МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)
Факультет компьютерных технологий и прикладной математики
Кафедра анализа данных и искусственного интеллекта

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3
ПО ДИСЦИПЛИНЕ «Технологии создания и поддержки программного обеспечения»
Тема: Техническое задание: Мобильное приложение «Gastronomix»

Комиссаренко Елизавета Евгеньевна
(Ф.И.О. студента)

студентка 22 группы 2 курса

Литвинов Давид Алексеевич
(Ф.И.О. студента)

студент 22 группы 2 курса

Направление подготовки 09.03.03 Прикладная информатика

Преподаватель факультета компьютерных технологий
и прикладной математики _____ Вишняков Р. Ю.
(Ф.И.О)

Краснодар 2026 г.

Цель работы: изучение и получение навыков создания и проверки технических заданий для программных продуктов.

Основные составные части технического задания

1. Цели и задачи проекта:

Цель: автоматизация учёта продуктов и индивидуальный подбор рецептов.

Задачи:

1. Организовать единую цифровую базу имеющихся продуктов.
2. Реализовать удобную систему регистрации и выбора предпочтений пользователя.
3. Разработать AI-ассистента для генерации рецептов на основе текущих продуктов.

2. Функциональные требования:

Описание пользовательских сценариев, интерфейсов и взаимодействия:

ФТ.01. Авторизация в системе

Функция обеспечивает создание и проверку учетных данных пользователя (через Email, Google, Apple или сторонние сервисы). Это необходимо для облачной синхронизации данных о продуктах между устройствами.

ФТ.02. Выбор пищевых предпочтений и ограничений

Возможность настройки персонального профиля, где указываются аллергии, диеты, особенности питания. Функция служит фильтром для всех последующих рекомендаций системы.

ФТ.03. Добавление продуктов в холодильник

Позволяет пользователю самостоятельно вносить данные о товаре: наименование, категория, количество и срок годности для формирования базы инвентаря.

ФТ.04. Сканирование штрих-кодов

Использование камеры устройства для считывания штрих-кода товара. Функция автоматически определяет название и тип продукта из глобальной справочной базы.

ФТ.05. Добавление продуктов по фото (компьютерное зрение)

Функция анализа изображений, которая распознает визуальные образы продуктов (овощи, фрукты, упаковки) и предлагает добавить их в список без ручного ввода наименований.

ФТ.06. Просмотр и анализ содержимого холодильника

Функция вывода полного списка продуктов с индикаторами свежести и анализом запасов по категориям.

ФТ.07. Составление рецептов из имеющихся продуктов

Алгоритм поиска в базе рецептов, который сопоставляет имеющиеся в наличии ингредиенты с необходимыми для блюда, предлагая варианты с учётом предпочтений пользователя.

ФТ.08. Персонализированный подбор рецепта с учётом истории и предпочтений

Модуль, который анализирует историю действий пользователя и его предпочтения для формирования индивидуальной ленты рекомендаций.

ФТ.09. Автоматическое составление меню на день/неделю

Функция планирования рациона, распределяющая имеющиеся продукты по приемам пищи таким образом, чтобы соблюдать нормы питания.

Создание пользовательских сценариев

Описание типичных процессов взаимодействия пользователя с вышеуказанными функциями:

Сценарий 1: первичная настройка и наполнение базы.

Пользователь проходит авторизацию (ФТ.01) и сразу переходит в профиль для выбора ограничений (ФТ.02). Затем он делает фотографии (ФТ.05) продуктов, принесенных из магазина или имеющихся в холодильнике. Система проводит распознавание (ФТ.06) и автоматически

классифицирует (ФТ.03) их в базе. В итоге в разделе просмотра содержимого (ФТ.06) пользователь видит актуальный цифровой список своих продуктов.

Сценарий 2: приготовление ужина.

Пользователь открывает приложение и использует функцию анализа содержимого (ФТ.06). Видя, что срок годности томатов истекает, он запускает составление рецептов (ФТ.07). Система предлагает «Томатный суп», выполняя персонализированный подбор (ФТ.08), допустим, пользователь указал без лука. После выбора блюда пользователь может использовать функцию планирования меню (ФТ.09), чтобы система записала приём пищи на вечер и расписала КБЖУ.

Сценарий 3: дистанционное пользование холодильником.

Находясь в магазине, пользователь сомневается, есть ли у него дома соус. Он открывает анализ содержимого (ФТ.06) дистанционно. Для проверки нового товара он использует сканирование штрих-кода (ФТ.04), чтобы мгновенно узнать, подходит ли данный продукт под его пищевые предпочтения (ФТ.02) и не дублирует ли он уже имеющиеся дома запасы.

3. Нефункциональные требования

Требования к производительности, безопасности, надежности, масштабируемости и удобству использования:

Производительность

Данные требования определяют скорость работы приложения и его способность справляться с вычислительными задачами:

Время отклика при поиске: время отображения списка продуктов при фильтрации или поиске по инвентарю не должно превышать 0,5 секунды.

Скорость распознавания: обработка фотографии и идентификация продуктов на ней должна занимать не более 3 секунд при наличии стабильного интернет-соединения.

Генерация рецептов: формирование персонализированной подборки рецептов должно осуществляться в пределах 5 секунд.

Пропускная способность: система должна поддерживать одновременную работу до 1000 активных пользователей без ожидания времени отклика сервера.

Безопасность

Требования к защите конфиденциальной информации и разграничению прав доступа:

Аутентификация пользователей: доступ к персональным данным и спискам продуктов должен быть возможен только после успешной авторизации через уникальный идентификатор логин/пароль или протокол авторизации (OAuth).

Защита персональных данных: все данные о пищевых ограничениях и аллергиях пользователей должны храниться и передаваться в зашифрованном виде (протокол HTTPS).

Приватность данных: система не должна передавать информацию о составе корзины продуктов пользователя третьим лицам без его явного согласия.

Безопасность сессий: автоматическое завершение неактивных сессий пользователя через 30 дней для предотвращения несанкционированного доступа с чужих устройств.

Надежность и масштабируемость

Определяют стабильность работы приложения и возможности его дальнейшего роста:

Коэффициент доступности: система должна быть доступна 99,9% времени (за исключением каких-либо технических работ).

Отказоустойчивость: при отсутствии интернет-соединения приложение должно обеспечивать просмотр уже загруженного списка продуктов и локальных рецептов в оффлайн режиме.

Целостность данных: в случае сбоя приложения при добавлении продукта система должна гарантировать сохранение целостности базы данных без дублирования записей.

Горизонтальная масштабируемость: архитектура серверной части должна позволять увеличивать вычислительные мощности при росте базы данных продуктов и рецептов без изменения программного кода.

Поддержка различных платформ: приложение должно корректно работать на операционных системах iOS и Android актуальных версий, обеспечивая одинаковый уровень функциональности.

4. Технические детали:

Анализ технологий:

Для разработки системы «Gastronomix» выбран современный технологический стек, обеспечивающий кроссплатформенность, высокую скорость обработки запросов и интеграцию с инструментами искусственного интеллекта.

Frontend (Клиентская часть): React + TypeScript + Tailwind CSS..

Выбор стека обусловлен необходимостью создания производительного и масштабируемого веб-интерфейса. Использование TypeScript обеспечивает типизацию данных и минимизирует ошибки при разработке. Приложение реализовано как PWA (Progressive Web App), что позволяет пользователю устанавливать его на смартфон и использовать функции камеры для сканирования продуктов. React-компоненты отвечают за динамическое отображение инвентаря, управление рецептами и интеграцию с API.

Backend (Серверная часть): Python + FastAPI.

Выбран в качестве высокопроизводительного серверного решения. Python обеспечивает гибкость разработки, а фреймворк FastAPI гарантирует минимальное время отклика при приеме и обработке фотографий и данных со смартфона.

Database & Auth (База данных и авторизация): PostgreSQL + SQLAlchemy.

В качестве системы управления базами данных используется PostgreSQL, развернутая в Docker-контейнере. Проектирование схемы данных включает сложные связи (Many-to-Many) между продуктами и рецептами, что требует написания оптимизированных SQL-запросов. Взаимодействие с базой на стороне бэкенда реализовано через SQLAlchemy (ORM). Система авторизации построена на протоколе OAuth 2.0 с использованием JWT-токенов, обеспечивая безопасный доступ и синхронизацию данных между сессиями пользователя.

AI Engine (Интеллектуальные модули): Google ML Kit + OpenAI GPT-4o Vision.

Для реализации функций распознавания используются два уровня технологий:

Google ML Kit Barcode Scanner: работает локально на устройстве (оффлайн), обеспечивая мгновенное считывание штрих-кодов за 0,5–1 сек.

OpenAI GPT-4o Vision: выступает в роли fallback-решения для анализа фотографий весовых продуктов (овощи, фрукты) или домашней еды, не имеющей маркировки.

Архитектурное проектирование

Продукт базируется на клиент-серверной архитектуре, где фронтенд-приложение (React) обеспечивает интуитивно понятный интерфейс для ввода данных, а высокопроизводительный бэкенд (FastAPI) выступает в роли интеллектуального центра обработки. В качестве хранилища используется

реляционная база данных PostgreSQL, обеспечивающая целостность и согласованность данных.

Описание архитектуры

Взаимодействие со штрих-кодом: клиентская часть с помощью специализированных библиотек захватывает изображение с камеры и считывает код. Полученные данные передаются на Backend через REST API. Сервер производит поиск продукта в базе данных PostgreSQL, сопоставляет его с глобальными справочниками и возвращает клиенту метаданные: название, категорию и рекомендуемый срок хранения.

Обработка визуальных образов: в случае отсутствия маркировки или штрих-кода, приложение передает изображение на уровень AI Engine **(**GPT-4o Vision**)**. Серверный модуль анализирует визуальный контент, классифицирует продукт и автоматически вносит его в инвентарь пользователя с сохранением истории распознавания.

Синхронизация и состояние: все действия пользователя (добавление продуктов, изменение личных предпочтений, управление запасами) обрабатываются сервером и сохраняются в централизованной БД. Это гарантирует мгновенную синхронизацию между сессиями и сохранность данных. Архитектура готова к масштабированию и интеграции дополнительных микросервисов.

5. Планы и этапы разработки

Планирование разработки

Проект разделен на ключевые этапы, ориентированные на последовательное изучение стека и интеграцию технологий:

Этап 1: проектирование и обучение

срок выполнения: 4 недели (май)

ключевой момент: изучение основ react и fastapi, согласование архитектуры базы данных postgresql и подготовка технического задания в перерывах между экзаменами.

Этап 2: настройка backend и базы данных

срок выполнения: 4 недели (июнь)

ключевой момент: разработка серверной логики на fastapi, создание структуры таблиц в postgresql через sqlalchemy и реализация системы авторизации.

Этап 3: frontend-разработка

срок выполнения: 4 недели (июль)

ключевой момент: реализация пользовательского интерфейса на react, настройка логики захвата изображения с камеры и интеграция фронтенда с серверным api.

Этап 4: интеграция ai engine и финализация

срок выполнения: 3 недели (август)

ключевой момент: подключение gpt-4o vision для анализа продуктов, тестирование точности распознавания и контейнеризация проекта в docker.

Этап 5: комплексное тестирование и отладка

срок выполнения: 1 неделя (конец августа)

ключевой момент: проведение интеграционных тестов фронтенда и бэкенда, проверка стабильности работы ии-модуля при различных условиях освещения и исправление выявленных ошибок перед релизом.

Распределение задач

Комиссаренко Елизавета: разработка клиентской части приложения на react, проектирование функциональных требований и интерфейса, реализация взаимодействия с камерой и API.

Литвинов Давид: разработка серверной части на fastapi, проектирование и администрирование базы данных postgresql, интеграция нейросетевых моделей для анализа визуальных данных.

6. Требования к тестированию

Определение критериев приемки

Критерии приемки определяют условия, при которых продукт «Gastronomix» считается готовым к внедрению и эксплуатации:

Функциональная полнота: все 9 функциональных требований, включая авторизацию, распознавание продуктов и генерацию рецептов, реализованы и работают корректно.

Стабильность AI-модулей: точность распознавания штрих-кодов через Google ML Kit составляет не менее 98%, а корректность идентификации продуктов через GPT-4o Vision не менее 85%.

Синхронизация данных: данные, внесенные с одного устройства, должны отображаться на другом устройстве под той же учетной записью Supabase.

Производительность: время отклика интерфейса и загрузки списков не превышает установленных нефункциональных требований.

Отсутствие критических ошибок: в системе отсутствуют баги такие, как вылеты приложения, потеря данных в облаке.

Создание тестовых сценариев

Для проверки качества продукта разработаны следующие сценарии тестирования:

Тест 1: проверка интеграции со сканером (Google ML Kit)

Действие: пользователь наводит камеру на штрих-код продукта питания через интерфейс приложения.

Ожидаемый результат: клиент на react считывает код, backend на fastapi находит продукт в базе данных postgresql, и в инвентаре появляется карточка товара с правильным названием.

Тест 2: тестирование визуального распознавания (GPT-4o Vision)

Действие: пользователь фотографирует огурец.

Ожидаемый результат: фотография передается на сервер, модель распознает объект как «огурец», определяет категорию «овощи» и предлагает пользователю подтвердить добавление в холодильник.

Тест 3: проверка синхронизации данных и авторизации

Действие: пользователь авторизуется через oauth, добавляет продукт «молоко» и выходит из системы. затем выполняет повторный вход в тот же аккаунт.

Ожидаемый результат: после авторизации список продуктов автоматически подгружается из базы данных через api, и продукт «молоко» присутствует в списке.

Тест 4: проверка генерации рецепта с учетом ограничений

Действие: в профиле указано ограничение «без лактозы». Пользователь запрашивает рецепт из имеющихся продуктов, например, курица, сливки, овощи.

Ожидаемый результат: система предлагает рецепты, не содержащие молочных сливок, или предлагает заменить их альтернативой, строго соблюдая фильтры, сохраненные в базе данных.

7.Процесс внедрения

Процесс внедрения программного обеспечения «Gastronomix» предполагает поэтапный вывод продукта в эксплуатацию и разделен на несколько ключевых направлений.

Во-первых, это развертывание серверной инфраструктуры, которое включает настройку docker-контейнеров с образом fastapi и организацию защищенного соединения с базой данных postgresql.

Во-вторых, это публикация клиентской части приложения в магазинах google play и app store, включая прохождение модерации и настройку метаданных.

В-третьих, это этап первичной миграции данных, в ходе которого в систему загружаются справочники категорий продуктов и базовый набор рецептов.

Заключительным этапом внедрения является проведение ознакомительного инструктажа для пользователей, который реализуется через систему интерактивных подсказок (onboarding) внутри интерфейса react, помогающих быстро освоить функции сканирования и работы с инвентарем.

8.Ограничения и риски

Разработка и эксплуатация системы сопряжены с рядом технических и юридических ограничений, которые учитываются при проектировании.

Технические риски включают возможную нестабильность работы внешнего api gpt-4o vision и зависимость качества распознавания продуктов от условий освещенности, качества камеры устройства и стабильности интернет-соединения. В случае отсутствия сети функции ии-анализа становятся временно недоступны. юридические ограничения проекта фиксируют отказ разработчиков от ответственности за любые медицинские последствия, связанные с употреблением пищи или возникновением аллергических реакций, так как приложение носит вспомогательный, а не консультационный характер.

Также существуют лимиты на количество запросов к нейросетевой модели в рамках бесплатного тестового периода, что может ограничить частоту использования функций распознавания одним пользователем.

9.Критерии приемки

Успешное завершение проекта и его готовность к внедрению определяются выполнением ряда формализованных критериев качества.

Во-первых, это функциональная полнота, означающая, что все заявленные в тз прецеденты (авторизация, сканирование штрих-кодов, распознавание по фото и генерация рецептов) работают без критических ошибок.

Во-вторых, это точность работы алгоритмов, где показатель верного распознавания продуктов из базового набора должен составлять не менее 90%.

В-третьих, это показатели производительности, при которых время отклика бэкенда на fastapi при стандартных запросах к базе данных не должно превышать 1.5 секунд.

Дополнительным критерием является кроссплатформенная совместимость, гарантирующая стабильное отображение интерфейса на react на различных типах мобильных устройств и корректное сохранение состояния корзины и инвентаря в базе данных postgresql после перезапуска приложения.

Контрольные вопросы

1. Что такое техническое задание?

Основной документ, содержащий требования заказчика к системе, в соответствии с которыми осуществляется создание и разработка конечного продукта.

2. Для кого составляется техническое задание?

ТЗ служит основой для планирования, разработки, тестирования и внедрения программного обеспечения, обеспечивая всем участникам проекта общее понимание того, что должно быть сделано.

3. Может ли техническое задание на разработку системы не содержать требований к тестированию, приемке? Если да, то в каком случае?

Теоретически да, если эти процессы описаны в отдельных регламентах, но методичка рекомендует включать их в ТЗ как обязательные критерии качества.

4. Какие основные блоки входят в построение технического задания?

Цели и задачи проекта, функциональные требования, нефункциональные требования, технические детали, планы и этапы разработки, требования к тестированию, процесс внедрения, ограничения и риски, критерии приемки.

5. Какие существуют этапы подготовительных работ по созданию технического задания?

Анализ требований, исследование и анализ рынка, определение целей и задач проекта, разработка функциональных требований, определение нефункциональных требований, выбор технологий и инструментов, разработка архитектуры системы, планирование разработки, разработка плана тестирования, оценка рисков и ограничений, документирование и согласование.

6. Может ли техническое задание быть составлено не в соответствии с ГОСТ?

Да, стандарт носит рекомендательный характер, но знание структуры ГОСТ 34.602-2020 является важным профессиональным навыком.

7. Приведите пример структуры технического задания по ГОСТ 34.602-2020.

Общие сведения, назначение и цели, характеристики объекта, требования к системе, состав работ, порядок контроля и приемки.

8. В чем основное структурное отличие ГОСТ 34.602-1989 и ГОСТ 34.602-2020?

Версия 2020 года более детализирована в вопросах информационной безопасности, современных технологий и процедур оценки соответствия.

9. Почему необходимо уметь составлять техническое задание?

Для минимизации ошибок при разработке, точного планирования сроков и юридической фиксации ожиданий заказчика.

10. Должны ли требования соответствовать определенным характеристикам? Если да, то каким?

Да, атомарность, понятность, проверяемость, непротиворечивость, идентифицируемость и независимость от реализации.

11. Для чего необходима идентификации требований в техническом задании?

Она нужна для присвоения каждому пункту уникального ID, чтобы:

Трассировать **(**отслеживать**):** связь требования с архитектурой, кодом и тестами.

Управлять правками**:** точно ссылаться на пункт при обсуждении, избегая путаницы.

Контролировать покрытие**:** видеть, что реализовано, а что нет.

12. Приведите примеры систем идентификации назовите их сильные и слабые стороны.

Порядковая нумерация вроде id001 проста в использовании, но не дает никакой информации о сути задачи что затрудняет поиск в объемных документах

Иерархическая система вида 1.1.2 наглядно показывает структуру разделов, но крайне неудобна при редактировании так как добавление одного пункта в середину списка сбивает всю нумерацию ниже

Мнемоническая идентификация с кодами типа fr для функциональных задач позволяет сразу понять тип требования и сохраняет id неизменным при любых правках текста, но требует предварительного описания всех правил кодирования для команды